

Рядки

Літерали рядків

Послідовність будь-яких символів, записана у одинарних або подвійних лапках, називається рядком (тип `str`). Літерали `"` (або `""`) презентують порожній рядок. Подвійні лапки зазвичай використовують, якщо серед символів, що складають рядок, є апостроф, наприклад: "П'ять – ціле число" (і навпаки, одинарні лапки використовують, якщо рядок містить подвійні лапки).

Як аргумент функції `print()` можна використовувати рядок у потрійних лапках (`"""` або `"""`), записаний у кілька рядків — тоді він буде виведений на екран теж у кілька рядків:

```
print('''Речення, яке ви бачите, записане  
у два рядки''')
```

Навіть якщо символи рядка утворюють число (`'100'`), дату (`'27.05.2019'`), IP-адресу (`'192.168.0.1'`) тощо, для інтерпретатора все це звичайний текст (послідовність символів) без жодного змісту.

Символи

Мова Python, на відміну від багатьох інших мов програмування, не має окремого символьного типу — для неї кожен поодинокий символ належить до типу `str`, тобто є рядком, який складається з одного символу.

Кожному символу поставлено у відповідність унікальне ціле число (код символу). Для перетворення символу в код і навпаки призначені вбудовані функції `ord()` і `chr()`:

```
code_A = ord('A')  
code_a = ord("a")  
char1041 = chr(1041)  
print(code_A, code_a, char1041)      # 1040 1072 Б
```

Керівні символи

Спеціальний символ `\n` інтерпретується як кінець рядка. Тобто, замість

```
print('''Речення, яке ви бачите, записане  
у два рядки''')
```

для отримання такого ж результату можна написати

```
print('Речення, яке ви бачите, записане\nу два рядки')
```

або, пригадавши, що кожне виведення функції `print()` завершується символом нового рядка:

```
print('Речення, яке ви бачите, записане')  
print('у два рядки')
```

Інший часто вживаний символ — символ горизонтальної табуляції `\t`:

```
print('1\t2\t3\t...')           # 1      2      3      ...
```

Керівні символи починаються з оберненої косої риски `\`, за якою слідує один чи кілька інших символів (уся послідовність символів трактується інтерпретатором як один символ). Обернена коса риска вказує на те, що наступний за нею символ (символи) мають трактуватися по особливому. Її часто використовують для екранування спеціальних символів (`\\`, `\'`, `\` та ін.), які без неї трактувалися би по іншому, наприклад:

```
print('П\'ять – ціле число')      # П'ять – ціле число  
print('one\\two\\three')         # one\two\tthree
```

Відсутність оберненої косої риски в першому випадку призвело би до появи синтаксичної помилки, у другому було би надруковано `one wo hree`.

Щоб уникнути екранування символів, в Python використовуються неформатовані рядки, які починаються символом `r`:

```
print(r'one\two\tthree')         # one\two\tthree
```

Доступ до елементів рядка. Індекси та зрізи

Кожен елемент рядка (символ) має індекс — унікальне ціле число, яке однозначно вказує на порядковий номер символа у рядку. Індекси можна

відраховувати з початку рядка (тоді нумерація починається з нуля: 0, 1, 2, ...), а можна з кінця (тоді індекси від'ємні: -1, -2, -3 і т. д.):

```
s = 'рядок'
print(s[0], s[2], s[-3])# р д д
```

Але слід пам'ятати, що рядок — незмінюваний тип даних, тобто намагання змінити, вставити або вилучити елемент рядка викличе помилку:

```
s[0] = 'P'
# TypeError: 'str' object does not support item assignment
```

Помилка виникне і у випадку, коли елемента з вказаним індексом не існує:

```
print(s[5])          # IndexError: string index out of range
```

Операція зрізу `[i:]` повертає частину рядка, починаючи з символу з індексом `i` та закінчуючи символом з індексом `j-1` (подібно до того, як генерувалася послідовність функцією `range()`):

```
s = 'рядок'
print(s[1:3])        # яд
print(s[:3])          # ряд (відсутність першого індекса означає "від початку")
print(s[1:])          # ядок (відсутність другого індекса означає "до кінця")
print(s[:])           # рядок ("від початку до кінця" — копія рядка)
```

Повний синтаксис зрізу містить три індекси: `[i:j:k]`. Третім індексом `k` задається крок, який за його відсутності приймається рівним 1:

```
print(s[1::2])        # яо (від елемента з індексом 1 до кінця з кроком 2)
```

У зрізах допускається використання від'ємних індексів, при цьому від'ємний крок вказує на вибір елементів справа наліво:

```
print(s[1:-2])        # яд (передостанній символ (з індексом -2) вже не включається)
print(s[3:0:-1])       # оя (перший елемент (з індексом 0) не включається)
print(s[-3:])          # док (три останні елементи)
```

Зрізи дають змогу змінювати рядки:

```
s = s[0].upper()+s[1:] # функція upper() перетворює символ у верхній регістр
print(s)               # Рядок
```

Конкатенація (об'єднання) та повторення рядків

Рядки можна об'єднувати, утворюючи таким чином нові рядки. Така операція називається конкатенацією і здійснюється за допомогою оператора +:

```
name = 'Роман'
s = 'Мене звати ' + name + '.'
print(s)      # Мене звати Роман.
```

Спроба об'єднати рядок з об'єктом іншого типу спричинить помилку, тому цей об'єкт має бути попередньо перетворений в рядок з використанням функції str():

```
name = 'Роман'
age = 47
s = 'Мене звати ' + name + '.'
s += ' Мені ' + str(age) + ' років.'
print(s)      # Мене звати Роман. Мені 43 роки.
```

Для об'єднання кількох однакових рядків (повторення певної послідовності символів) використовують оператор *:

```
s = '-'*5 + 'abc'*2 + '-'*5
print(s*3)  # -----abcbcb-----abcbcb-----abcbcb-----
```

Форматування рядків

Крім конкатенації, Python пропонує й інші способи формування/форматування рядків. Історично першим є просте позиційне форматування з використанням оператора %, але воно вважається застарілим і тому не розглядатиметься.

У 3-й версії Python з'явилася можливість форматування з використанням функції format(). У наведеному нижче коді цифрами у фігурних дужках позначені порядкові номери аргументів функції (за потреби після двокрапки передається специфікатор формату):

```
s = 'Мене звати {}'.format(name)
s = "Мене звати {0}. Мені {1:d} роки.".format(name, age)
s = "Мене звати {my_name}. Мені {my_age} роки.".format(my_name=name, my_age=age)
```

Цей спосіб також підтримує параметри форматування, які вказуються після символу ":". Тип вирівнювання задається символами "<" (за лівим краєм), ">" (за правим краєм), "^" (по центру), після яких вказується ширина поля:

```
from random import randint
```

```
for i in range(3):
    n = randint(1, 20)
    print('{0:<3}{1:>21}'.format(n, '+'*n))
```

Параметри форматування дійсних чисел такі ж, як і для оператора %, наприклад:

```
import math
print('{0:.3f}'.format(math.pi))          # 3.142
```

Починаючи з версії Python 3.6, доступні форматовані літерали рядків (f-рядки), які дають змогу підставляти не тільки значення змінних, але й значення виразів. Інтерпретатор Python розпізнає f-рядок за символом f перед відкриваючими лапками¹:

```
name, current_year, year_of_birth = 'Роман', 2023, 1976
s = f"Мене звати {name}. Мені {current_year - year_of_birth} роки."
```

Є принаймі ще один спосіб форматування — шаблонні рядки. Проте, оскільки останні не є ключовими властивостями мови, а містяться у модулі string стандартної бібліотеки Python, то особливості їх використання тут не розглядаються.

Функція len()

```
s = 'Це рядок'
len(s)          # поверне 8 - кількість символів у рядку
```

Методи рядків

```
s = 'Це рядок'
s.upper()        # переводить символи у верхній регістр ('ЦЕ РЯДОК')
s.lower()        # переводить символи у нижній регістр ('це рядок')
s.capitalize()   # перший символ у верхній регістр ('Це рядок')
s.replace('ок', 'ки') # заміна усіх входжень 'ок' на 'ки' ('це рядки')
s.find('ряд')     # повертає індекс першого входження 'ряд' (3)
s.count('ряд')    # повертає кількість входжень 'ряд' (1)
s.isalpha()       # перевіряє, чи рядок містить тільки літери (False)
s.isdigit()       # перевіряє, чи рядок містить тільки цифри (False)
s.startswith('Це') # перевіряє, чи рядок починається з 'Це' (True)
s.endswith('.')    # перевіряє, чи рядок закінчується крапкою (False)
```

¹ Після двокрапки також можна передавати специфікатори формату

```
s.strip()                # вилучає пробіли на початку та в кінці рядка
```

Важливо! Оскільки рядки є незмінюваними типами, то методи рядків повинні бути частиною інструкцій присвоєння. Так, у наступному коді рядок `s` не зміниться (інтерпретатор просто створить новий об'єкт, але не прив'яже його до змінної):

```
s = 'Це рядок'
s.replace('ок', 'ки')
print(s)                # Це рядок
```

Щоб усе спрацювало, код має бути таким:

```
s = 'Це рядок'
s = s.replace('ок', 'ки')
print(s)                # Це рядки
```

Але:

```
s = 'Це рядок'
print(s.replace('ок', 'ки')) # Це рядки
print(s)                    # Це рядок
```

Окремо виділимо методи перетворення рядків у списки і навпаки. Метод `split()` повертає об'єкт іншого типу — список підрядків. За замовчуванням рядок розбивається на підрядки по символу пробілу, але аргументом методу можна задати будь-який інший розділювач, який не обов'язково складається з одного символу:

```
sentence = 'Я прочитав про програмування мовою Python'
words = sentence.split()
print(words)          # ['Я', 'прочитав', 'про', 'програмування', 'мовою', 'Python']
pro = sentence.split('про')
print(pro)             # ['Я ', 'читав ', ' ', 'грамування мовою Python']
```

Використовуючи метод `split()` та індекси, можна написати компактний код, наприклад, для отримання розширення файлу з його імені:

```
filename = 'фото_1.jpg'
print(filename.split('.')[1])# jpg
```

Метод `join()`, у свою чергу, утворює новий рядок шляхом конкатенації елементів списку, який передається аргументом, при цьому рядок, до якого застосовується метод, виступає розділювачем:

```
c = '--'.join(words)
```

```
print(c)                # Я--прочитав--про--програмування--мовою--Python
```

Перевірка на входження та посимвольний обхід рядка

Часто (в умовних конструкціях) потрібно мати відповідь на питання, чи містить рядок деякий підрядок. Для цього у логічних виразах використовують оператор `in`:

```
s = 'Це рядок'
s1 = 'ряд'
print(s1 in s)          # True
```

Обхід елементів рядка (символів) здійснюється з використанням інструкції `for`:

```
s = 'Це рядок'
for symb in s:
    print(symb.upper(), end = '')
# буде надруковано ЦЕ РЯДОК
```

У наведеному вище коді змінна `symb` по чергово набуває значень символів рядка ('Ц', 'е', ' ', 'р' і т. д.), а у тілі циклу ці значення друкуються у верхньому регістрі.

Посимвольний обхід рядка може здійснюватися і з використанням індексів:

```
for i in range(len(s)):
    print(s[i].upper(), end = '')
```

або

```
i = 0
while i < len(s):
    print(s[i].upper(), end = '')
    i += 1
```
